



Fehler in Python

Bestimmt sind Ihre Programme schon einmal mit einer Fehlermeldung abgestürzt, wie zum Beispiel die folgende:

```
SyntaxError: invalid syntax
```

Als Nutzer ist es natürlich sehr ärgerlich, wenn das Programm ständig mit solchen Meldungen abstürzt (und man eventuell auch nichts damit anfangen kann).

Python bietet daher die Möglichkeit, mit Fehlern "elegant" umzugehen:

```
try:
    x = int(input("Bitte geben Sie eine Zahl ein: "))
except ValueError:
    print("Sie müssen eine Zahl eingeben!")
else:
    print("Das ist eine gute Zahl!")
finally:
    print("Danke für die Eingabe.")
```

Es wird versucht, den Code im try-Block auszuführen. Sollte dabei ein ValueError auftreten, stürzt das Programm nicht ab, sondern führt den Code im except-Block aus. Falls kein Fehler auftritt, wird der Code im else-Block ausgeführt. Der Code im finally-Block wird in jedem Fall zum Schluss ausgeführt.

Eine Übersicht über die verschiedenen Fehlerarten finden Sie zum Beispiel bei w3schools¹. Einige der häufigsten Errortypen sind z. B. ValueError, KeyError und KeyboardInterrupt.



Aufgabe 1: Sichere Division

Schreiben Sie ein Programm, was zwei Zahlen vom Nutzer einliest. Stellen Sie dabei sicher, dass der Nutzer auch wirklich Zahlen eingibt.

Berechnen Sie anschließend den Quotienten der beiden Zahlen. Achtung: Man darf nicht durch 0 teilen!

¹https://www.w3schools.com/python/python_ref_exceptions.asp

Aufgabe 2: Ein- und Ausgabe

Schreiben Sie ein Programm, das einen Dateipfad vom Nutzer einliest und anschließend versucht, die Datei zu öffnen und den Text "Hallo Welt!" hineinzuschreiben. Verhindern Sie, dass Ihr Programm abstürzt, falls die Datei nicht existiert. Denken Sie daran, dass Sie auch im Fehlerfall die Datei wieder schließen müssen!

Aufgabe 3: Fehlerprotokoll

Legen Sie eine Liste mit drei von Ihnen festgelegten Zahlen an. Lassen Sie anschließend in einer Schleife einen zufälligen Index zwischen 0 und 10 generieren. Versuchen Sie, die Zahl an der generierten Stelle auszugeben - falls der Index zu groß ist, loggen Sie einen Fehler in eine Datei. Die Schleife soll abbrechen, sobald alle Zahlen in der Liste ausgegeben wurden.

Bonusaufgabe 4: Eigene Fehlermeldungen

Mit dem Keyword `raise` können Sie selbst Fehler erzeugen.

Ein Programm soll zur Notenverwaltung eingesetzt werden. Schreiben Sie eine Funktion `pruefe_note(note)`, die überprüft, ob der Parameter `note`...

- (a) ...eine ganze Zahl ist
- (b) ...im Bereich 1-6 liegt.

Falls nicht, erzeugen Sie einen Fehler geeigneten Typs.

Schreiben Sie anschließend ein Hauptprogramm, das Noten vom Nutzer einliest und diese mithilfe Ihrer Funktion validiert. Wenden Sie dabei eine geeignete Fehlerbehandlung an.